Missing Callers Under a Response Budget: Finishing Versus Finding in a Local Gemma 4 Agent

A prespecified omission probe and follow-ups in 30 Python libraries

Arjun Thilak R, AI Researcher, Eros Innovation

Ramkumar MV, AI Researcher, Eros Innovation

Kaggle Writeup, Gemma 4 Developer Agent Paper Track, November 2026

Abstract. Code-graph tools give repository agents incomplete caller lists. We test whether a local Gemma 4 31B agent still finds a caller its graph answer leaves out. In a prespecified probe over 30 Python libraries (750 targets, 11,250 primary episodes), the answer omits one caller whose call ran in the library's own tests and keeps every other entry byte-identical. Under an 8-response limit, the incomplete list raised recovery of the omitted caller by 15.6 points over no list (95% CI 11.5 to 19.8), the opposite of our prespecified expectation, though a pilot had pointed this way. Follow-ups implicate finishing: one harness line before the last response ("This is your last response. Call submit_answer now.") added 22.5 points over a neutral line, and with it the list's advantage shrank to 0.3 points. Finding stayed hard: with 98.0% of incomplete-list episodes submitting, the missing caller still cost 10.5 points against the complete list, and with twice the budget most harmed episodes stopped with responses to spare (post hoc). Warnings, lexical evidence and ledgers did not significantly help, and every main-run submission claimed completeness. In 5 more local models, withholding the caller always cost recovery, but the list's sign tracked what agents did with it: those that kept searching gained, while Nemotron and the small Gemma 4 E4B and E2B submitted the incomplete list almost at once and lost up to 63.2 points. Evaluations should score finishing and finding separately.

1. Introduction

Caller lists given to repository agents are often incomplete: whole-program analyses are unsound by design [R6], static Python call graphs miss callees seen at run time [R7], and runtime traces see only tested paths. For 136 of our 750 targets, LocAgent's own graph answer [R1] already misses a test-exercised caller.

Bellam finds that much of the gain from LocAgent-style tools for small agents comes from the form of the submission [R18], and Xu finds call-site recall limited by thoroughness rather than retrieval [R24]; neither withholds a known caller. We add a controlled test: delete one test-exercised caller from an otherwise byte-identical graph answer, then change how the agent's last response is handled. This measures finishing and finding separately. For `google/gemma-4-31B-it` we contribute:

- An omission probe with call-site truth and no-list and complete-list baselines (§3, §4).
- Evidence that an incomplete list helps mainly by making the agent finish, while the missing caller's cost persists when nearly every episode submits (§6).
- Tests across libraries, budgets, task framings and models: the omission cost recurred in every model; the list effect reversed in agents that submitted the list at once (§6.6, §7).
- Exposure accounting and a CPU-verifiable release of every episode (§2, §8).

The featured competition scores bugs fixed by Gemma 4 agents that use a graph tool under a response budget. A repair agent that stops early or trusts an incomplete caller list leaves call sites unedited, so our results bear on how such agents are budgeted and evaluated; §5.1 tests the competition's checkpoint and §6.6 its small models (in BF16).

2. Exposure: is an offered graph tool requested?

In our earlier pvlib repair study with the same model ([R14]; records in the release), `graph_query` and `source_reference_search` were offered in 270 episodes and requested zero times, while bash ran 6,093 times.

In this study's tool-choice arm (exploratory; 150 targets, seed 11), the agent called an offered `graph_query` tool in 47/150 episodes, and in 150/150 when the task recommended it (Fig. 1).

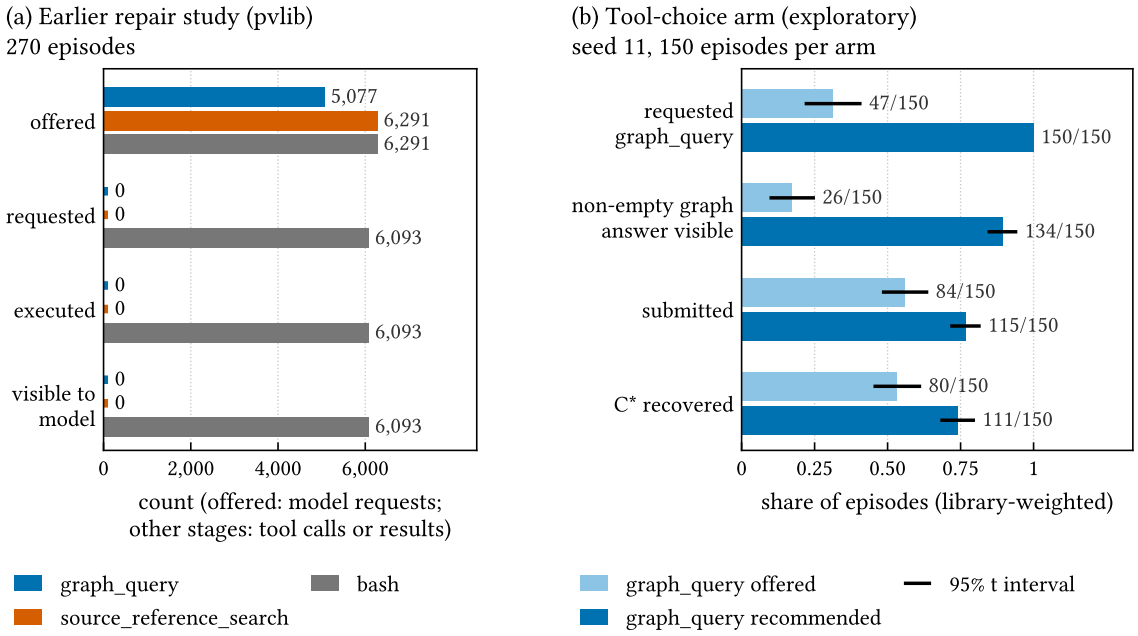


Figure 1. Exposure: (a) earlier repair study, by stage and tool; (b) tool-choice arm.

3. The omission probe

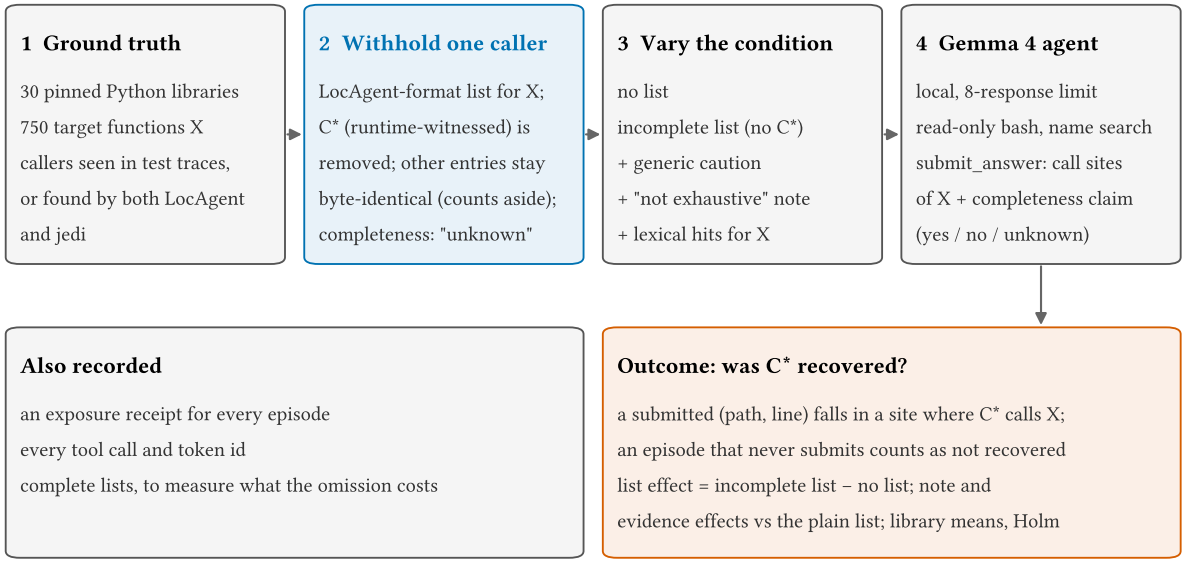


Figure 2. Probe design: truth, the omitted caller, conditions, agent and outcome.

Libraries and truth. We pinned 30 permissively licensed Python libraries whose offline test suites pass (≥95% of tests); the pilot used two others. For a target function X, the verified callers V are those seen in runtime traces of the library's own tests (`sys.monitoring` [R15]) plus those that both LocAgent's graph [R1] and jedi report; a caller with a trace site is runtime-witnessed. Truth is recorded per call site, and a caller that only one static backend reports is never omitted.

Items. An eligible target is a plain package function or method with 2–12 verified callers, at least one runtime-witnessed, whose rendered answers fit untruncated and reveal no omitted call site (3,590 of 23,911 package functions). A frozen hash selects 25 per library (750 items) and, for each, one runtime-witnessed caller C* to omit. The production graph code renders the answer (cap 12,000 bytes); removing C* and its snippet leaves every other entry byte-identical apart from counts, which unit tests check.

Conditions and agent. Table 1 lists the conditions. The agent, `google/gemma-4-31B-it` in BF16 [R17], runs on vLLM at temperature 0.2, thinking off. It has read-only bash in a jail, a lexical search tool and `submit_answer` for every site where another package function invokes X (path and line) with a completeness claim (yes, no or unknown). The system prompt announces its 8-response limit (a response is one model message with at least one tool call); the context holds 16,384 tokens; an episode that uses every response without submitting ends as NoAnswer.

Table 1. What the agent sees in each main-block condition (plan codes in parentheses). Answers are JSON listing each caller's file, line span and code excerpt, with completeness "unknown"; a hash picks this wording or a paraphrase per item.

Condition	Answer block	Added text
No list (C0)	none	none
Incomplete list (C1)	caller list without C*	none
Caution (C2)	incomplete list	generic caution ("Be careful with this answer…") + action sentence
Not-exhaustive note (C3)	incomplete list	"This index answer is not an exhaustive list of callers. A missing relationship does not show that no other caller exists." + action sentence
Lexical block (C4a)	incomplete list	lexical search hits for X's name, at most 20 hits and 2,000 bytes

Action sentence: "Inspect the relevant source and search for additional references before you submit."

Outcome and analysis. Recovery, the primary outcome, means that some submitted (path, line) falls inside a site where C* invokes X; an episode that never submits counts as not recovered. The main block runs every item in every condition at seeds 11, 23 and 47. The confirmatory contrasts are the list effect (incomplete list minus no list; H3 in the plan), the note effect (not-exhaustive note minus caution; H1) and the evidence effect (lexical block minus incomplete list; H2). Paired item differences are averaged over seeds, then within libraries, then equally over the 30 libraries; a two-sided t test over library means gives p, Holm-adjusted across the contrasts [R16]. The plan set a meaningful effect of 10 points and expected the note and evidence effects to be positive and the list effect negative; inference applies to this purposive set of libraries. Secondary blocks at seed 11 (complete lists holding all of V, LocAgent's own answers, tool choice) are descriptive.

Freeze and integrity. The analysis plan (`protocols/EXPA_PREREG.md`, sha256 57991a7e) and its hash lock were committed at 2026-11-01 04:02:27 UTC, before any main-library episode. The pilot changed no parameter; its list effect, printed in the plan, was already positive (25.8 points). All 13,740 design cells (11,250 in the main block) completed and 0 were lost to infrastructure; exposure receipts confirm every answer and lexical block. Independent post-results code reproduced 921/921 frozen primary values.

4. Confirmatory results

Under guaranteed exposure, a caller list that omits one caller raised that caller's recovery by 15.6 points over no list (95% CI 11.5 to 19.8; Holm p 4.4e-08), opposite to the prespecified direction. Recovery ranged from 56.8% with no list to 74.6% with the lexical block (Fig. 3).

Neither added statement nor added evidence showed a significant benefit. The not-exhaustive note changed recovery by -1.5 points relative to generic caution (-3.5 to 0.5; Holm p 0.172), and the lexical block by 2.2 points relative to the incomplete list alone (-0.3 to 4.7; Holm p 0.172). Both intervals exclude 10-point effects but do not establish equivalence; sign-flip tests and library bootstraps agree (Fig. 4).

Prespecified secondary outcomes. In every condition the agent searched for X's name in at least 99.3% of episodes, yet every main-block submission claimed that its list was complete (8,101 of 8,101; 9,968 of 9,968 across all blocks), although each answer block stated completeness "unknown".

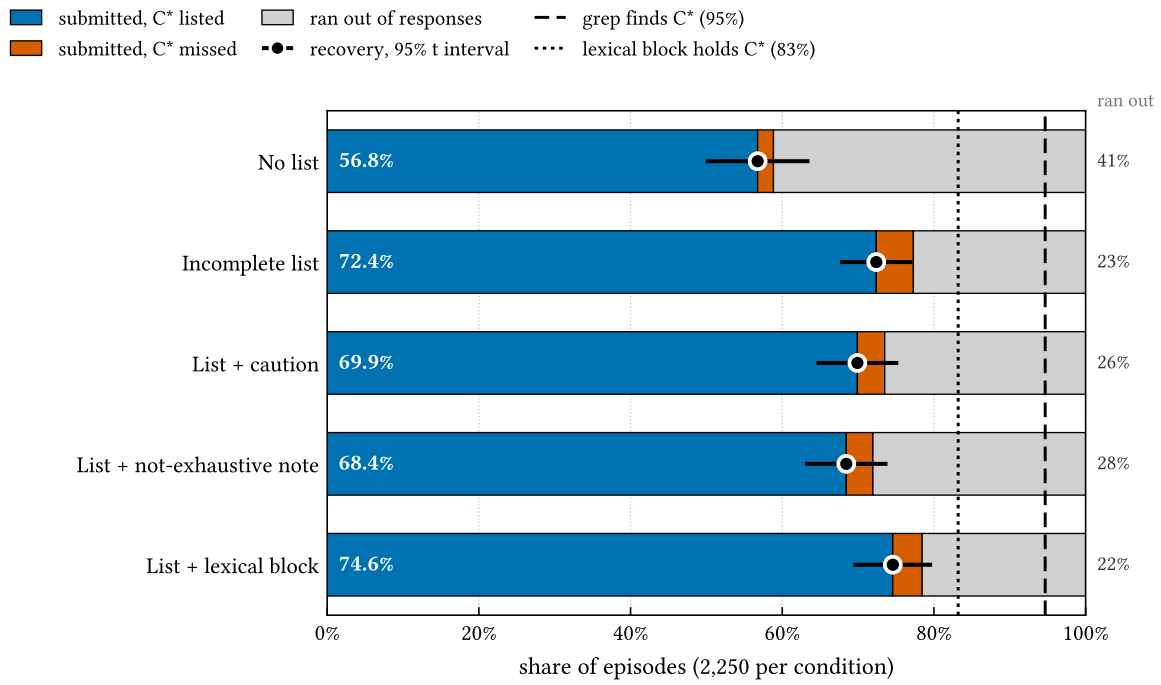


Figure 3. Main block by condition: exits (bars), recovery of C* with 95% t intervals (points) and model-free references (lines).

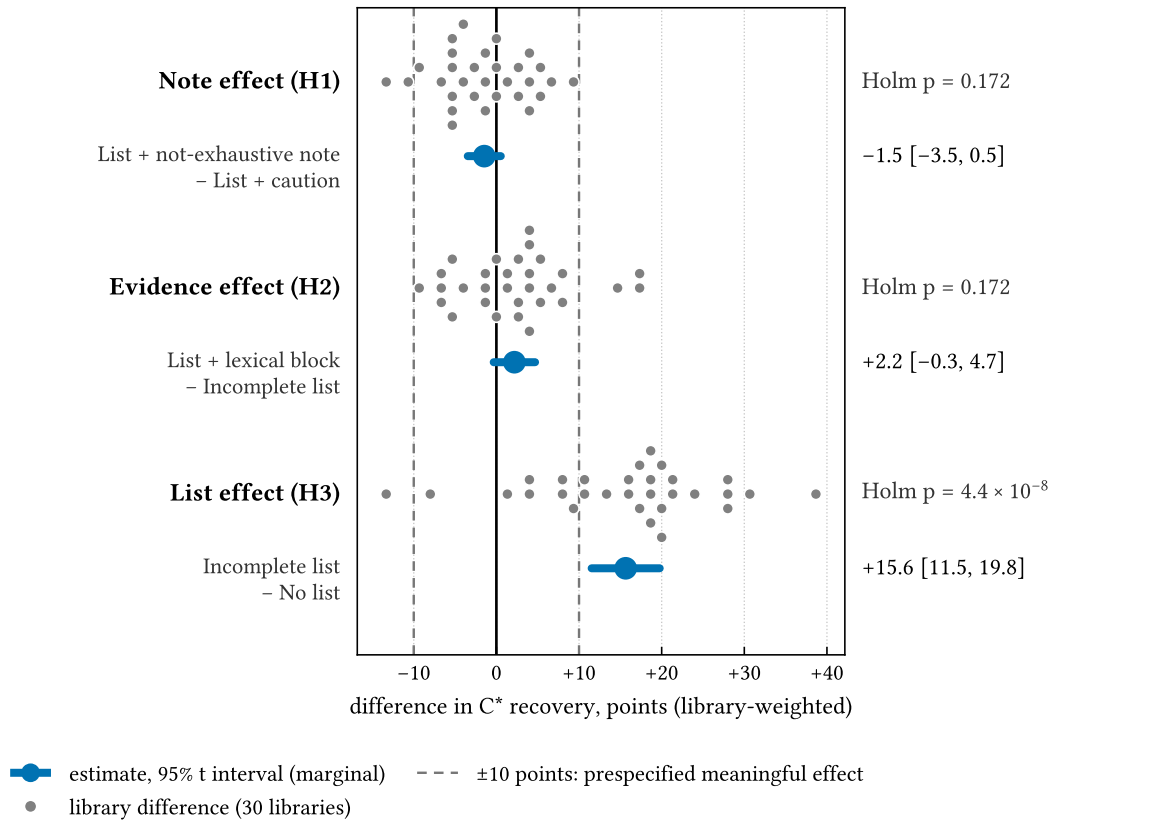


Figure 4. Confirmatory effects with 95% t intervals; dots are libraries; dashed lines at ±10 points.

5. Exploratory analyses

Designed after the primary results were opened, this section is descriptive (nominal intervals) apart from the tests in §5.1 and §5.2.

Exits. Every episode ended as Submitted or NoAnswer; none hit the context limit. The incomplete list raised submission by 18.4 points (14.0 to 22.9), while recovery given submission stayed at 93%–96% in every main-block condition (Fig. 3).

5.1 Other checkpoints

Under protocols frozen before their own results, `google/gemma-4-12B-it` (BF16) and the main track's quantization-aware checkpoint `gemma-4-31b-it-qat-w4a16-ct` repeated the whole design. The list effect was 14.8 points (11.6 to 17.9; Holm p <0.001) and 9.6 (5.3 to 13.9; Holm p <0.001), respectively.

5.2 Callers missed by name search

This run rebuilt the items so that no call site of C* contains X's name: 236 items in 27 libraries, all dynamic calls, missed by name search, the lexical block and LocAgent's answer. In its prespecified test, the list raised recovery from 7.3% to 14.2% and submission from 32.6% to 55.2%, a list effect of 7.0 points (1.8 to 12.2; p 0.010 as prespecified, Holm-adjusted 0.031).

6. Finishing versus finding: prespecified follow-ups

Each follow-up was frozen before its own episodes with its own prespecified primary tests (release supplement S5o–S5af); post hoc contrasts carry nominal intervals, and Holm adjustment across these tests changes 0 decisions.

One traitlets item (seed 11) shows both failures. With no list the agent printed a call site of C* (`Container.subclass_init`) in its first response but ran out of responses; with the incomplete list it submitted in time, claimed "complete" and missed C*; with the complete list it submitted every caller.

6.1 Telling the agent it is on its last response

Rerunning no list and incomplete list (seed 11), we added one harness line before the final response: none, a neutral line ("Your previous tool call was processed normally.") or a deadline line ("This is your last response. Call submit_answer now."). Averaged over both lists, the deadline line beat the neutral one by 22.5 points (18.8 to 26.2; p <0.001) and no line by 20.3; submission reached 94.8%–98.1% (Fig. 5), and the list's advantage fell from 17.1 points under the neutral line to 0.3 (-2.4 to 3.0). The conservative precision bound, the share of submitted entries at verified call sites, fell to 92.6%–94.6% (from 97.5%–97.9%).

A factorial follow-up (seed 23) found that the information ("This is your last response.") did most of the work; adding the instruction to it changed recovery by 0.2 points (post hoc).

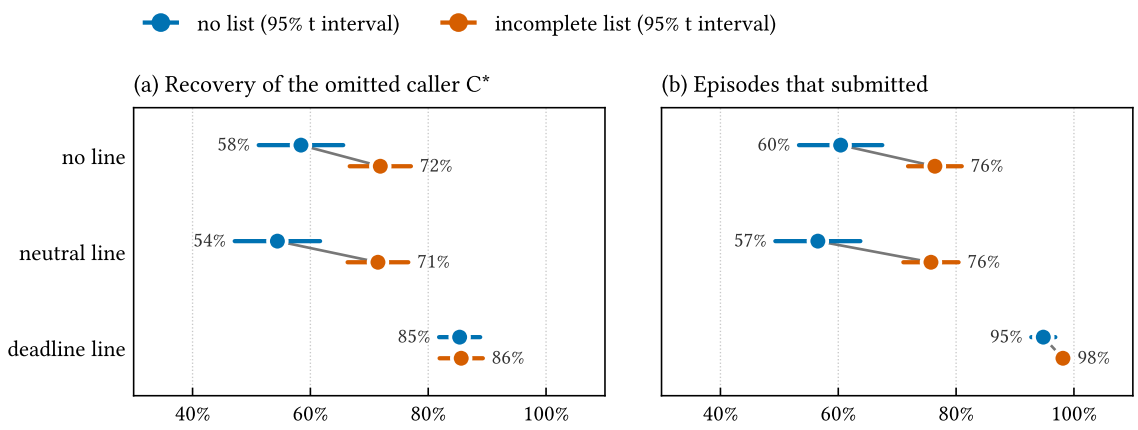


Figure 5. No list and incomplete list, seed 11: recovery and submission by final-turn line, 95% t intervals.

6.2 What the missing caller still costs

Under the deadline line, complete and incomplete lists ran at seeds 61, 73 and 89. Withholding C* cost 10.5 points (96.1% versus 85.6%) although 98.0% of incomplete-list episodes submitted; counting every unfinished one as recovered still leaves 8.5 points (5.8 to 11.2; post hoc). In 259 item-seed pairs only the complete list recovered C* (the reverse in 22); in 184 of these no tool output had displayed a C* call site (strict rule). Scored at function level, the cost is unchanged (11.0 points).

6.3 Ledgers that did not help

Harness-built ledgers of call sites the agent had already printed showed no significant effect on the omission cost at the last response (p 0.108; reached by 1,159 of 4,500 episodes) or after a first submission with a chance to revise (p 0.129), and rarely held the missing site.

6.4 A doubled budget

Doubling the response limit and the context, with the deadline line before the new last response, cut the omission cost from 10.5 to 8.3 points (unrounded difference 2.3, 0.8 to 3.8; p 0.005; across runs). Only 9.2% of incomplete-list episodes reached the new last response, and in 148 of 201 pairs where only the complete list recovered C*, the incomplete-list episode submitted with responses to spare (post hoc), as reported for coding agents [R18, R21].

6.5 An impact-analysis task

Reframed as impact analysis (X's signature will change; find every place to update; seed 11) on the same items and scorer, both prespecified effects recurred: the incomplete list beat no list by 5.3 points (2.2 to 8.5) and withholding C* cost 9.9 (6.3 to 13.4; both Holm $p \le 0.002$). Against the original framing at the same seed, the omission cost changed by only -1.2 points (-4.8 to 2.4; across runs).

6.6 Other models

Prespecified after the Gemma results, A8 ran no list, incomplete list and complete list at seed 11, then no list and incomplete list with the deadline line, on other local models in the same harness. Of 17 models tried, 12 could not drive the harness or failed the prespecified dev check (5 wrote malformed tool calls, 4 had requests rejected, 3 failed otherwise). Figure 6 gives the prespecified tests of the rest (Holm across all; supplement S5ae–S5af).

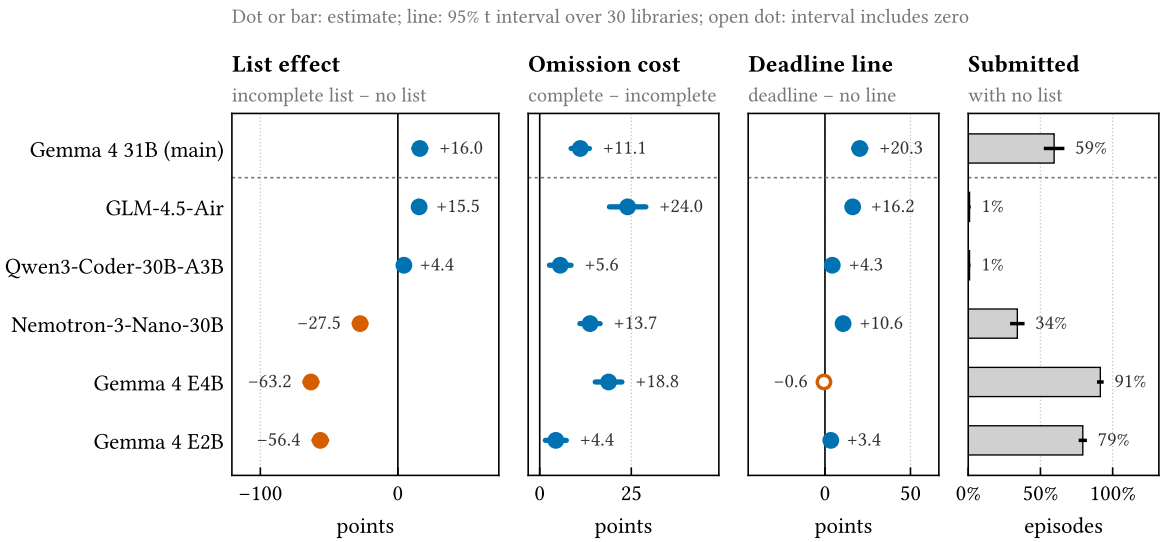


Figure 6. Effects by model (points, seed 11) and submission without a list; Gemma 4 31B from the main run and §6.1.

Withholding the caller cost recovery in every model (all Holm $p \le 0.008$). The list helped models that kept searching. Nemotron, E4B and E2B submitted it almost at once (at least 98.3% of episodes) and lost: E4B recovered 8.1% with the incomplete list against 71.3% without one. With the complete list it recovered only 26.9%, because it submitted callers' line spans rather than call lines. The deadline line helped every model but E4B, which rarely ran out of responses; GLM kept its list advantage under it (16.9 points), unlike Gemma 4 31B.

> **Design rules.** Log tool requests, exposure, submission and recovery separately; make the last response explicit, as budget trackers do [R27]; expect little from warnings or ledgers, which showed no significant benefit here; check completeness claims against evidence; test small models against a no-tool baseline, since a list can end a search that would have succeeded. An incomplete tool answer can also raise scores by helping the agent finish, so compare no-tool and complete-answer baselines under the same budget and last-response handling, or rewards and leaderboards will credit finishing as finding.

7. Generality and scope

For Gemma 4 31B the list effect was positive in 28 of 30 libraries, the last-response line in 30 and the omission cost in 28 (post hoc); pooled estimates kept their direction across callers that name search misses (§5.2), seeds, budgets and task framings (§6). Across models (§5.1, §6.6), the omission cost recurred everywhere, the deadline line helped wherever agents ran out of responses, and the list effect's sign depended on whether the agent kept searching after it received the list. The method can be adapted to any list-returning tool with independently witnessed positives, such as references, search hits or test selections: establish the positives, delete one from an otherwise byte-identical answer, and measure exposure, submission and recovery separately.

Prompts, tools and the response limit are ours; the study is Python only, with English prompts, and none of it tests repair success. Runtime witnesses cover only tested paths, and the reference set V misses callers that neither the tests nor both backends find; 36 of 939 C* sites rest on a name match alone, and no recovery depends only on such a site. Most omitted callers C* are direct calls (708 of 750); dynamic callers stay hard (14.2% recovery with the incomplete list), as in LLM call-graph inference [R25]. The list effect is a bundled effect: prompt length, the retained callers and their framing change together. Freezes are repository commits, not third-party registrations. The main-track repositories

rich and httpx overlap ours; we used our own commits and no competition data.

8. Release and reproducibility

Every result number is a registry key pointing to a hash-pinned file or a derivation; a checker re-renders the text and fails on any number without a key. The release (Apache-2.0; upstream excerpts keep their licences) holds the items with call-site truth, the harness and prompts, every episode record, the analysis code, a CPU verifier that re-runs every frozen analysis, and a notebook that recomputes the central estimates in seconds. One command, `python -I -B reproduce/verify.py`, runs every check. Code: https://github.com/arjunthilak05/missing-callers-gemma4; dataset package: https://www.kaggle.com/datasets/rarjunthilak/absent-edges-expa-v1.

9. Related work

Code-graph tools for agents are established [R1, R2, R3, R22], and their non-use is known [R3, R5]: in CodeCompass, 58% of trials with graph access made no graph-tool calls [R4]. Language models miss omitted information and treat partial answers as complete [R8, R9, R20]; agents overclaim complete work [R19]. Agents, Gemma 4 among them, adopt corrupted tool returns [R10], though disclaimers and explicit failure states help [R11, R23]; CodeAnchor argues, without a controlled omission, that agents tolerate missing edges [R3]. Execution-witnessed call edges exist at scale [R7, R12], and repograph explains absences [R13]. Omission probes exist for documents [R8] and for safety facts in tool responses [R26]; to our knowledge, none withholds a known true caller from a code-graph answer.

Disclosure

Only locally served open models were evaluated as agents.

References

- R1. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL 2025. doi:10.18653/v1/2025.acl-long.426. Code: https://github.com/gersteinlab/LocAgent at 4935b557326c154bad8e8dcf3747cc8d32d1f387.
- R2. Ouyang et al. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. ICLR 2025. arXiv:2410.14684.
- R3. Lin, Zhou, Yang, Li. How Much Static Structure Do Code Agents Need? A Study of Deterministic Anchoring. arXiv:2606.26979v2, 2026.
- R4. Paipuru. CodeCompass: Navigating the Navigation Paradox in Agentic Code Intelligence. arXiv:2602.20048, 2026.
- R5. Faghih et al. Tool Preferences in Agentic LLMs are Unreliable. EMNLP 2025. doi:10.18653/v1/2025.emnlp-main.1060.
- R6. Livshits et al. In Defense of Soundiness: A Manifesto. CACM 58(2), 2015. doi:10.1145/2644805.
- R7. Bouzenia, Krishan, Pradel. DyPyBench: A Benchmark of Executable Python Software. Proc. ACM Softw. Eng. 1(FSE), 2024. doi:10.1145/3643742.
- R8. Fu et al. Absence Bench: Language Models Can't See What's Missing. NeurIPS 2025 Datasets and Benchmarks. doi:10.52202/085713-1277.
- R9. Min et al. When Absence Is Evidence: Evaluating Completeness-Sensitive Negative Reasoning in Large Language Models. arXiv:2608.04591, 2026.
- R10. Yang et al. Agents' Overreliance on Unreliable Tools. arXiv:2609.05587v2, 2026.
- R11. Sun et al. Tools Fail: Detecting Silent Errors in Faulty Tools. EMNLP 2024. doi:10.18653/v1/2024.emnlp-main.790.
- R12. Li et al. An Execution-Verified Multi-Language Benchmark for Code Semantic Reasoning (TraceEval). arXiv:2605.11006, 2026.

- R13. Chahwan. repo-graph, bench/absence. https://github.com/James-Chahwan/repo-graph/tree/0f8ecfa1cc103af030ef2438ca50e9634a30290e/bench/absence.
- R14. mini-swe-agent, https://github.com/SWE-agent/mini-swe-agent at 04d809ceab9df28f9adaed04488418015917 Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.
- R15. Shannon. PEP 669: Low Impact Monitoring for CPython. https://peps.python.org/pep-0669/.
- R16. Holm. A Simple Sequentially Rejective Multiple Test Procedure. Scand. J. Statist. 6:65–70, 1979.
- R17. Google. gemma-4-31B-it model card. https://huggingface.co/google/gemma-4-31B-it.
- R18. Bellam. How Do Small Coding Agents Use Code Graphs and Embeddings? arXiv:2610.33958, 2026.
- R19. Smyth et al. Quantifying Overclaiming Propensity in Frontier LLM Agents. arXiv:2609.20812, 2026.
- R20. Guan et al. Beware of Reasoning Overconfidence: Pitfalls in the Reasoning Process for Multi-solution Tasks. arXiv:2512.01725, 2025.
- R21. Chen et al. Knowing When to Stop: The Evidence-Action Gap in Coding Agents. arXiv:2610.31895, 2026.
- R22. Yu et al. CodeNib: A Multi-View Data System for Serving Repository Context to Coding Agents. arXiv:2607.25431, 2026.
- R23. Sethi et al. Fabrication After Tool Failure: Tool-Augmented Agents Assert Values Their Tools Did Not Return. arXiv:2609.14758, 2026.
- R24. Xu. Does a Language Server Save Tokens for Coding Agents? A Measurement Methodology and Preliminary Study. arXiv:2608.13568, 2026.
- R25. Venkatesh et al. An Empirical Study of Large Language Models for Type and Call Graph Analysis in Python and JavaScript. Empirical Software Engineering 30(6):167, 2025. doi:10.1007/s10664-025-10704-3.
- R26. Zhang et al. EpiSEM-Bench: Evaluating Safety and Evidence Recovery in LLM Agents under Missing Evidence. arXiv:2610.31650, 2026.
- R27. Liu et al. Budget-Aware Tool Use Enables Effective Agent Scaling. arXiv:2511.17006, 2025.